

# Structured Computer Architecture

## Chapter 3.13: Harvard Architecture

### Fifth-Order Systems (5-OS)

Based on Structured Computer Architecture: An Order-Based Approach

# Outline

# Learning Objectives

By the end of this chapter, you will understand:

- ▶ Fifth-Order Systems (5-OS) and the Harvard abstract model
- ▶ Historical development from Turing machines to Harvard Mark I
- ▶ Differences between Harvard and Von Neumann architectures
- ▶ RISC principles and their advantages
- ▶ Cache memory hierarchy and performance implications
- ▶ Interpreting vs. executing instruction paradigms
- ▶ Modern implementations and design considerations

# From Fourth to Fifth-Order Systems

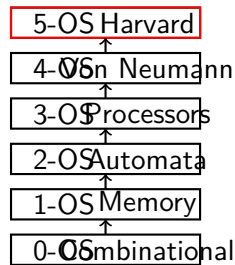
## System Evolution Recap:

- ▶ **0-OS:** Combinational circuits (no feedback)
- ▶ **1-OS:** Memory circuits (first feedback loop)
- ▶ **2-OS:** Automata (second feedback loop)
- ▶ **3-OS:** Elementary processors (third feedback loop)
- ▶ **4-OS:** Von Neumann computers (fourth feedback loop)
- ▶ **5-OS:** Harvard computers (fifth feedback loop)

## Fifth Loop Characteristics:

- ▶ Processor + Program Memory = 4-OS
- ▶ Adding Data Memory creates 5-OS
- ▶ Separate memory spaces for instructions and data

## System Order Progression



# Theoretical Foundation: Universal Turing Machine

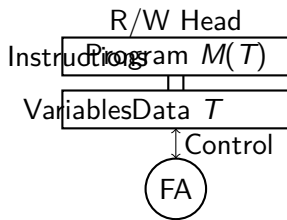
## Turing Machine Inspiration:

- ▶ Alan Turing divided UTM tape into two parts:
  - ▶ Machine description  $M(T)$  (program)
  - ▶ Associated tape  $T$  (data)
- ▶ Two-memory concept: programs and data
- ▶ Multi-tape TM: multiple memories, multiple heads

## UTM Processing Model:

- ▶ Collaboration between program and finite automaton
- ▶ Data changes through:
  1. Readings from program part
  2. State transitions of finite automaton
- ▶ Complexity factors:
  - ▶ Program size (flexible)
  - ▶ Automaton size (fixed in hardware)

## UTM Tape Organization



# Historical Timeline: Harvard Mark I (1944)

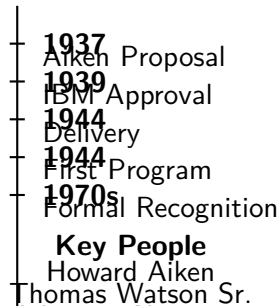
## Key Historical Facts:

- ▶ **March 29, 1944:** John von Neumann runs first program on Harvard Mark I
- ▶ **Architect:** Howard Aiken (Harvard University)
- ▶ **Manufacturer:** IBM (Endicott plant, New York)
- ▶ **Concept Origin:** November 1937 (Aiken's proposal to IBM)
- ▶ **Funding Approval:** February 1939 (Thomas Watson Sr.)

## Design Innovation:

- ▶ Two separate memories: programs and data
- ▶ Relay-based electromechanical computer
- ▶ General-purpose computational capability
- ▶ Inspired by need to solve differential equations

## Harvard Mark I Timeline



# Von Neumann's "First Draft" (1945)

## Document Background:

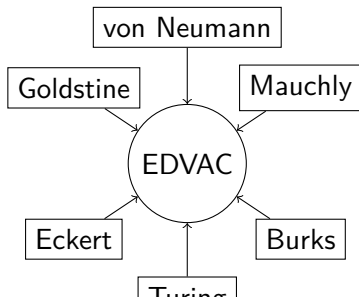
- ▶ **Title:** "First Draft of a Report on the EDVAC"
- ▶ **Author:** John von Neumann (sole credited author)
- ▶ **Context:** EDVAC development at Moore School
- ▶ **Distribution:** 24 typed copies by Herman Goldstine

## Key Contributors:

- ▶ **Herman Goldstine:** Typed and distributed document
- ▶ **John Mauchly:** ENIAC/EDVAC development
- ▶ **J. Presper Eckert:** Hardware design expertise
- ▶ **Arthur Burks:** Theoretical contributions
- ▶ **Alan Turing:** Theoretical foundation (UTM)

## EDVAC Development Team

### Collaborative Effort



# Architectural Comparison

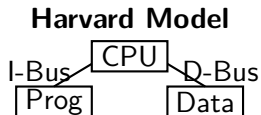
## Harvard Abstract Model:

- ▶ **Memory Organization:** Separate program and data memories
- ▶ **Access Pattern:** Simultaneous instruction and data access
- ▶ **Bus Structure:** Independent instruction and data buses
- ▶ **Performance:** Higher throughput, no memory conflicts
- ▶ **Complexity:** More complex memory system

## Von Neumann Abstract Model:

- ▶ **Memory Organization:** Unified memory for programs and data
- ▶ **Access Pattern:** Sequential access to instructions and data

## Model Comparison





# Interpreting vs. Executing Instructions

## Two Paradigms for Instruction Processing:

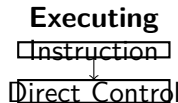
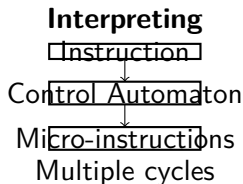
### Interpreting Instructions:

- ▶ Instructions processed by control automaton
- ▶ Sequence of micro-instructions generated
- ▶ Complex control logic with many states
- ▶ Flexible but slower execution
- ▶ Higher hardware complexity for control

### Executing Instructions:

- ▶ Instructions executed in single clock cycle
- ▶ Instruction fields directly control components
- ▶ Zero-state automaton (no control states)
- ▶ Faster execution, simpler control
- ▶ UTM with zero internal states concept

### Execution Models



# Reduced Instruction Set Computer (RISC)

## RISC Philosophy:

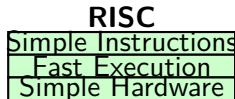
- ▶ Keep only simplest and most frequently used instructions
- ▶ Reject complex instructions that can be implemented as sequences
- ▶ Simple hardware design outperforms complex designs
- ▶ Machine code only marginally larger than CISC

## Performance Benefits:

- ▶ Faster execution due to simpler operations
- ▶ Better pipeline efficiency
- ▶ Reduced hardware complexity
- ▶ Easier verification and testing

## Design Principles:

### RISC vs CISC



CISC Performance Advantage

# Hardware-Software Segregation

## Clear Segregation Principle:

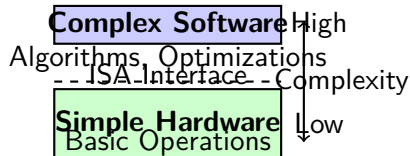
- ▶ **Simple Hardware:** Built from simple circuits
- ▶ **Complex Software:** Handles algorithmic complexity
- ▶ **Separation of Concerns:** Hardware does what it does best
- ▶ **Optimization:** Each layer optimized independently

## Hardware Simplicity:

- ▶ Most components are simple circuits
- ▶ Only decoder section has moderate complexity
- ▶ Size comparable to description size
- ▶ Easier specification, design, verification, testing

## Software Complexity:

### Segregation Model



# Cache Memory Levels

## Three-Level Cache Hierarchy:

- ▶ **L1 Cache:** Smallest, fastest (8KB-128KB)
- ▶ **L2 Cache:** Medium size, moderate speed (1MB-8MB)
- ▶ **L3 Cache:** Largest, slower but faster than main memory (8MB-64MB+)

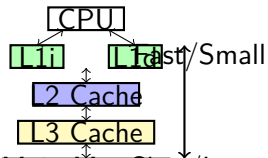
## L1 Cache Characteristics:

- ▶ On-chip, same die as processor
- ▶ Operates at processor speed
- ▶ Harvard split: L1i (instructions) + L1d (data)
- ▶ Provides fastest access to critical data

## Cache Hierarchy Principles:

- ▶ **Locality Principle:** Temporal and spatial locality

## Memory Hierarchy



# Cache Performance and Locality

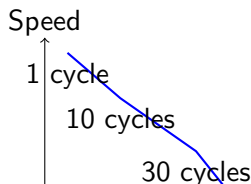
## Locality Principles:

- ▶ **Temporal Locality:** Recently accessed data likely to be accessed again
- ▶ **Spatial Locality:** Nearby data likely to be accessed soon
- ▶ **Sequential Locality:** Instructions executed in sequence
- ▶ **Loop Locality:** Repeated access to same code/data

## Cache Performance Metrics:

- ▶ **Hit Rate:** Percentage of accesses found in cache
- ▶ **Miss Rate:** Percentage of accesses not in cache
- ▶ **Hit Time:** Time to access cache on hit

## Cache Performance



# Modern Harvard Implementations

## Contemporary Applications:

- ▶ **Microcontrollers:** Embedded systems, IoT devices
- ▶ **DSP Processors:** Digital signal processing applications
- ▶ **GPU Architectures:** Parallel processing units
- ▶ **RISC Processors:** High-performance computing systems

## Implementation Challenges:

- ▶ **Memory Bandwidth:** Dual memory systems require more bandwidth
- ▶ **Synchronization:** Coordinating instruction and data access
- ▶ **Cache Coherency:** Maintaining consistency across caches

## Modern Implementation

Modern CPU

# Chapter Summary

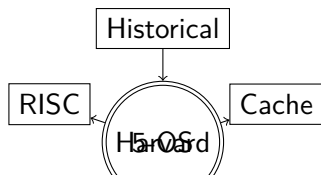
## Key Concepts Covered:

- ▶ Fifth-Order Systems (5-OS) and Harvard abstract model
- ▶ Historical development from Turing machines to modern computers
- ▶ Comparison between Harvard and Von Neumann models
- ▶ RISC principles and hardware-software segregation
- ▶ Cache memory hierarchy and performance implications
- ▶ Modern implementation considerations and challenges

## Architectural Innovations:

- ▶ Separate instruction and data memories

## Harvard Model Evolution



## Discussion Questions

1. How does the Harvard model address the von Neumann bottleneck?
2. What are the trade-offs between Harvard and von Neumann architectures?
3. How do RISC principles complement the Harvard architecture?
4. What role does cache hierarchy play in modern Harvard implementations?
5. How has the Harvard model influenced modern processor design?
6. What are the implications of separate instruction and data memories?
7. How do modern applications benefit from Harvard architecture principles?
8. What future developments might further evolve the Harvard model?